PROYECTO FINAL
Análisis de Ventas y Visualización de Datos
Tamalería “Don Rutilio”

Diplomado en Gestión y Visualización de Datos

con soluciones de Google Cloud

Descripción del proyecto
Reporte de análisis de datos enfocado en la integración, limpieza, análisis exploratorio y visualización de información de ventas, con el objetivo de generar insights que apoyen la toma de decisiones estratégicas y el crecimiento sostenible del negocio.

Elaborado por:
JESUS MONTSERRAT ZARAGOZA CARRILLO

Fecha: 07/02/2026

Índice

Problema del negocio..... 2
Objetivos del análisis de datos. 2
Datos utilizados..... 2
Proceso de análisis 2
Preparación y limpieza de los datos en Google Colab 3
Integración y análisis en la nube con SQL 15
Transformación y construcción de la base final 15
Decisiones técnicas..... 15
Preguntas de negocio 17
Insights del análisis de datos 18
- Volumen de ventas e ingresos 18
- Ganancias mensuales 18
- Zonas con mayor venta 18
- Producto (sabor) más vendido..... 19
- Categoría más vendida 19
- Métodos de pago..... 19

Conclusiones 19
Recomendaciones..... 19
Referencia:..... 20

Problema del negocio.

El pequeño negocio familiar dedicado a la venta de tamales ha experimentado un crecimiento sostenido; sin embargo, la información generada por sus operaciones (ventas, costos, producción e inventarios) se encuentra distribuida en diversos archivos y formatos. Esta falta de integración dificulta su análisis, limita la visibilidad del desempeño operativo y financiero, y obstaculiza la identificación de productos más rentables o de periodos en los que se presentan pérdidas por falta de inventario.

Objetivos del análisis de datos.

Centralizar y analizar la información en la nube con el fin de transformar los datos en información confiable y accionable que permita mejorar la toma de decisiones. En particular, se busca identificar la rentabilidad por producto por periodo, optimizar la gestión de inventarios y costos, reducir pérdidas operativas y apoyar un crecimiento ordenado y sostenible del negocio.

Datos utilizados.

Los datos utilizados corresponden al proyecto final de la Tamalería *"Don Rutilio"*, desarrollado como parte del diplomado en Gestión y Visualización de Datos con soluciones de Google Cloud. El conjunto de datos contiene información detallada de las ventas del negocio, registrada a nivel de renglón por artículo vendido.

El paquete de datos incluye información almacenada en múltiples formatos y fuentes, entre los que se encuentran archivos Excel (.xlsx), CSV (.csv), TXT (.txt) y una base de datos SQLite (.sqlite). Esta diversidad de formatos refleja un entorno real de negocio, en el cual la información se genera y almacena de manera heterogénea.

En cuanto a la estructura, cada renglón representa un producto vendido dentro de una transacción. El campo *id_venta* identifica una venta realizada por un cliente y puede repetirse cuando una misma transacción incluye varios productos. Las principales variables incluyen fecha y hora de la venta, punto de venta, producto, categoría, canal de venta, método de pago y precio.

Respecto a la calidad de la información, los datos presentan inconsistencias intencionales en nombres y formatos, así como el uso de separadores irregulares en los archivos CSV y TXT. Asimismo, existen patrones de comportamiento del negocio que no son evidentes a simple vista y requieren análisis exploratorio para ser identificados. Por ello, es indispensable realizar procesos de integración, limpieza, estandarización y validación de los datos antes de llevar a cabo el análisis y la generación de visualizaciones e insights para la toma de decisiones.

Proceso de análisis

El proceso de análisis de datos se desarrolló siguiendo un flujo de trabajo estructurado, con el objetivo de transformar información de ventas dispersa y heterogénea en un conjunto de datos confiable y apto para el análisis descriptivo, exploratorio y la generación de insights. Este proceso incluyó las etapas

de preparación, limpieza, transformación e integración de los datos, utilizando Google Colab, Google Cloud Storage y BigQuery sobre infraestructura de Google Cloud.

Preparación y limpieza de los datos en Google Colab

En una primera etapa, se utilizó Google Colab como entorno de trabajo para la preparación y limpieza de los datos provenientes de múltiples fuentes y formatos, tales como archivos Excel, CSV y TXT. A través de Python y librerías especializadas para el análisis de datos, se realizó la lectura de archivos con distintos separadores, la estandarización de nombres de columnas y la corrección de formatos inconsistentes.

Durante esta fase se atendieron problemas de calidad de la información, incluyendo registros incompletos, valores duplicados y variaciones en la escritura de variables categóricas. Asimismo, se validaron los tipos de datos, especialmente en campos críticos como fechas, precios y categorías de producto, con el fin de asegurar la coherencia y confiabilidad de la información antes de su integración.

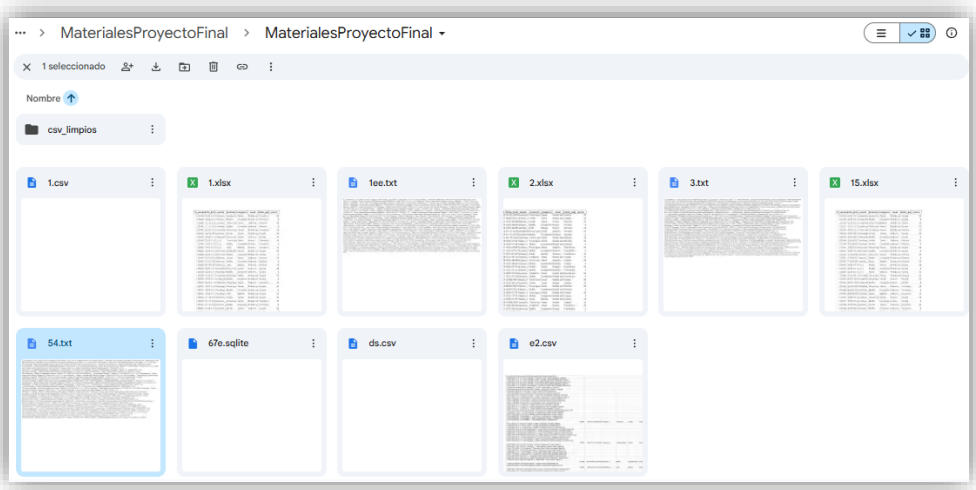
Una vez concluida la limpieza, los datos fueron consolidados en un único conjunto estructurado y exportados en un formato compatible para su posterior carga y análisis en la nube.

1. Se define las librerías.

Defición de Librerias

```python
import os
import pandas as pd
import sqlite3
import re
import openpyxl
from google.colab import auth
auth.authenticate_user()
from google.colab import drive
from google.cloud import storage
drive.mount('/content/drive')
import glob
```

2. Leer archivos almacenados en drive y guardarlos en la carpeta csv_limpios una vez transformado, posteriormente preparar el bucket con el nombre del proyecto, nombre del bucket y el destino.



Configuración de archivos y bucket

```python
CARPETA_LOCAL = "/content/drive/MyDrive/Proyecto Google/MaterialesProyectoFinal/MaterialesProyectoFinal"
CARPETA_CSV = os.path.join(CARPETA_LOCAL, "csv_limpios")
os.makedirs(CARPETA_CSV, exist_ok=True)

project_ID="tamalesdonrutilio"
BUCKET_NAME = "bucket_rutilio"
Destino = "datos/Base_general.csv"
```

3. Creación del bucket.

```python
client = storage.Client(project=project_ID)

# Obtener lista de buckets
buckets = [b.name for b in client.list_buckets()]

if BUCKET_NAME in buckets:
    print(f"El bucket '{BUCKET_NAME}' ya existe.")
else:
    bucket = client.bucket(BUCKET_NAME)
    bucket.storage_class = "STANDARD"

    bucket.create(location="us-central1")

    print(f" Bucket '{BUCKET_NAME}' creado correctamente.")
```

```
Bucket 'bucket_rutilio' creado correctamente.
```

4. Homologación de archivos con distinto formato a CSV y renombrar las bases.

Homologación de formato a csv y nombre de las bases

```python
# Contador
contador_base = 1  # Para Base1, Base2, Base3...

# ----------------------------
# PROCESAMIENTO
# ----------------------------
for archivo in os.listdir(CARPETA_LOCAL):
    ruta = os.path.join(CARPETA_LOCAL, archivo)

    if os.path.isdir(ruta):
        continue

    try:
        archivo_lower = archivo.lower()

        # ---------------- CSV ----------------
        if archivo_lower.endswith(".csv"):
            with open(ruta, 'r', encoding='utf-8', errors='replace') as f:
                lineas = [l.strip() for l in f if l.strip()]

            def split_line(line):
                line = line.strip('"')
                return re.split(r'\t|;\s*|,\s*|\|\s*', line)

            data = [split_line(l) for l in lineas]

            max_cols = max(len(row) for row in data)
            for row in data:
                row.extend([None] * (max_cols - len(row)))
```

```python
    encabezado = data[0]
    df = pd.DataFrame(data[1:], columns=encabezado, dtype=str)

# ---------------- TXT ----------------
elif archivo_lower.endswith(".txt"):
    with open(ruta, 'r', encoding='utf-8', errors='replace') as f:
        lineas = [l.strip() for l in f if l.strip()]

    def split_line(line):
        line = line.strip('"')
        return re.split(r'\t|;\s*|\|\s*', line)

    data = [split_line(l) for l in lineas]

    max_cols = max(len(row) for row in data)
    for row in data:
        row.extend([None] * (max_cols - len(row)))

    encabezado = data[0]
    df = pd.DataFrame(data[1:], columns=encabezado, dtype=str)

# ---------------- XLSX ----------------
elif archivo_lower.endswith(".xlsx"):
    hojas = pd.read_excel(ruta, sheet_name=None, dtype=str)

    for _, df in hojas.items():
        break  # solo la primera hoja

# ---------------- SQLITE / DB ----------------
elif archivo_lower.endswith(".sqlite") or archivo_lower.endswith(".db"):
    conn = sqlite3.connect(ruta)
    tablas = pd.read_sql(
        "SELECT name FROM sqlite_master WHERE type='table';",
        conn
    )
```

```python
            tabla = tablas['name'][0]
            df = pd.read_sql(f"SELECT * FROM {tabla}", conn, dtype=str)
            conn.close()

        else:
            print(f"Tipo no soportado: {archivo}")
            continue

        # ---------------- LIMPIEZA COMÚN ----------------
        df.columns = (
            df.columns
            .str.strip()
            .str.lower()
            .str.replace(" ", "_")
            .str.replace(r"[^a-z0-9_]", "", regex=True)
        )

        for col in df.columns:
            df[col] = df[col].astype(str).str.strip()

        # ---------GUARDAR CSV y homologar nombre de las base ----------
        nombre_csv = f"Base{contador_base}.csv"
        ruta_destino = os.path.join(CARPETA_CSV, nombre_csv)

        df.to_csv(ruta_destino, index=False, encoding="utf-8")
        print(f"Creado: {nombre_csv}")

        contador_base += 1

    except Exception as e:
        print(f"Error con {archivo}: {e}")

print("Todos los archivos convertidos a CSV limpio")
```

5. Se anexan las bases de datos por que las columnas comparten el mismo nombre de las columnas.

Anexar los archivos separados (Base General)

Se toma la decisión de Anexar por que las bases de datos comparten las misma columnas.

```python
# Ruta de tus CSV
path = "/content/drive/MyDrive/Proyecto Google/MaterialesProyectoFinal/MaterialesProyectoFinal/csv_limpios/*.csv"
files = glob.glob(path)

dfs = []
for file in files:
    df = pd.read_csv(file)
    dfs.append(df)

# Concatenamos todo
df_general = pd.concat(dfs, ignore_index=True)
print("Tamaño total de la base general:", df_general.shape)
df_general.head()
```

Información General.

```python
df_general.info()
```

Son 250000 datos con las columnas Id_venta , fecha_hora, puesto, categoría, canal, método_pago y precio.

```python
# Ruta de tus CSV
path = "/content/drive/MyDrive/Proyecto Google/MaterialesProyectoFinal/MaterialesProyectoFinal/csv_limpios/*.csv"
files = glob.glob(path)

dfs = []
for file in files:
    df = pd.read_csv(file)
    dfs.append(df)

# Concatenamos todo
df_general = pd.concat(dfs, ignore_index=True)
print("Tamaño total de la base general:", df_general.shape)
df_general.head()
```

```
Tamaño total de la base general: (250000, 8)
```

	id_venta	fecha_hora	puesto	producto	categoria	canal	metodo_pago	precio
0	100001	07-02-2025 11:17 AM	Observatorio_1	Tamal verde	Tamal	Pedido anticipado	Tarjeta	25
1	100002	06/07/2025 10:27	SantaFe_2	Verde	Tamal	Pedido anticipado	Tarjeta	25
2	100003	"2025-08-05 07:45:00"	Pantitlan_1	Verde	Tamal	Puesto	"Efectivo"	25
3	100003	2025-08-05 07:45:00	Pantitlan_1	Bolillo	Complemento	Puesto	Efectivo	6
4	100003	2025-08-05 07:45:00	Pantitlan_1	Café	"Bebida"	Puesto	Efectivo	12

Información General.

```python
df_general.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 250000 entries, 0 to 249999
Data columns (total 8 columns):
 #   Column       Non-Null Count   Dtype
---  ------       --------------   -----
 0   id_venta     250000 non-null  object
 1   fecha_hora   250000 non-null  object
 2   puesto       250000 non-null  object
 3   producto     250000 non-null  object
 4   categoria    250000 non-null  object
 5   canal        250000 non-null  object
 6   metodo_pago  250000 non-null  object
 7   precio       250000 non-null  object
dtypes: object(8)
memory usage: 15.3+ MB
```

6. Se revisa el tipo de formato de las columnas quitando duplicados en este caso la columna de fecha tenía alguno de estos formatos: “01/08/2025, “01/10/2025”, 2025/10/12, 2025-08-05,07-02-2025 11:17 AM

Analizacion de variables

```python
print("----Fecha_hora----")

puestos_total = df_general["fecha_hora"]

puestos_unicos = (
    puestos_total
    .dropna()
    .drop_duplicates()
    .sort_values()
)

display(puestos_unicos)
```

```
----Fecha_hora----
```

	fecha_hora
10446	"01/07/2025 08:10"
112620	"01/07/2025 09:06"
100490	"01/07/2025 10:59"
17911	"01/07/2025 11:40"
110563	"01/08/2025 06:16"
...	...

7. Análisis de la columna de puesto nombre mal homologados todos terminan con _1 o _2 y con “” ejemplo: “CU_1”.

```python
print("----Puestos----")

puestos_total = df_general["puesto"]

puestos_unicos = (
    puestos_total
    .dropna()
    .drop_duplicates()
    .sort_values()
)

display(puestos_unicos)
```

```
----Puestos----
```

	puesto
390	"CU_1"
30	"CU_2"
195	"CentroHistorico_1"
13	"Condesa_1"
19	"Coyoacan_1"
204	"DelValle_1"

8. La columna de producto de categoría y canal se puede observar que no esta homologada las palabras y tiene “”.

```python
print("----Producto----")
puestos_total = df_general["producto"]

puestos_unicos = (
    puestos_total
    .dropna()
    .drop_duplicates()
    .sort_values()
)

display(puestos_unicos)
```

```
----Producto----
```

	producto
263	"Atole caliente"
215	"Atole"
34	"Bolillo"

```python
print("----canal----")

puestos_total = df_general["canal"]

puestos_unicos = (
    puestos_total
    .dropna()
    .drop_duplicates()
    .sort_values()
)

display(puestos_unicos)
```

```
----canal----
```

	canal
15	"Delivery"
9	"Pedido anticipado"
77	"Puesto"
5	Delivery
0	Pedido anticipado
2	Puesto

dtype: object

```python
print("----metodo_pago----")

puestos_total = df_general["metodo_pago"]

puestos_unicos = (
    puestos_total
    .dropna()
    .drop_duplicates()
    .sort_values()
)
display(puestos_unicos)
```

```
----metodo_pago----
```

	metodo_pago
2	"Efectivo"
15	"Tarjeta"
54	"Transferencia"
3	Efectivo
0	Tarjeta
10	Transferencia

dtype: object

9. Se hace limpieza de estas inconsistencias encontradas.

Limpieza de Datos

```
# Quitar comillas en las columnas

import pandas as pd

df_general = df_general.applymap(lambda x: x.replace('"', '') if isinstance(x, str) else x)

df_general.head(10)
```

```
/tmp/ipython-input-4065005016.py:5: FutureWarning: DataFrame.applymap has been deprecated. Use DataFrame.map instead.
  df_general = df_general.applymap(lambda x: x.replace('"', '') if isinstance(x, str) else x)
```

	id_venta	fecha_hora	puesto	producto	categoria	canal	metodo_pago	precio
0	100001	07-02-2025 11:17 AM	Observatorio_1	Tamal verde	Tamal	Pedido anticipado	Tarjeta	25
1	100002	06/07/2025 10:27	SantaFe_2	Verde	Tamal	Pedido anticipado	Tarjeta	25
2	100003	2025-08-05 07:45:00	Pantitlan_1	Verde	Tamal	Puesto	Efectivo	25
3	100003	2025-08-05 07:45:00	Pantitlan_1	Bolillo	Complemento	Puesto	Efectivo	6
4	100003	2025-08-05 07:45:00	Pantitlan_1	Café	Bebida	Puesto	Efectivo	12
5	100004	07-22-2025 08:09 AM	RomaNorte_1	Tamal verde	Tamal	Delivery	Efectivo	25
6	100004	07-22-2025 08:09 AM	RomaNorte_1	Bolillo	Complemento	Delivery	Efectivo	6
7	100005	08-24-2025 08:38 AM	Lindavista_1	Tamal rojo	Tamal	Pedido anticipado	Efectivo	26
8	100006	2025-07-06 10:18:00	Tlalpan_1	Tamal gourmet	Tamal	Pedido anticipado	Efectivo	35
9	100006	2025-07-06 10:18:00	Tlalpan_1	Bolillo	Complemento	Pedido anticipado	Efectivo	6

10. Se hace limpieza de la columna de Fecha para homologar y dejar un solo formato para ello se localizan los distintos formatos.

```
df_general["fecha_hora"].head()
```

	fecha_hora
0	07-02-2025 11:17 AM
1	06/07/2025 10:27
2	2025-08-05 07:45:00
3	2025-08-05 07:45:00
4	2025-08-05 07:45:00

dtype: object

```
df_general.loc[df_general["fecha_hora"].isna(), "fecha_hora"].head(20)
```

fecha_hora

dtype: object

```
df_general["fecha_hora"].notna().sum()
```

```
np.int64(250000)
```

```python
#" Lipizar Fechas"

# ========================
# 0. Normalizar texto
# ========================
fecha_raw = (
    df_general["fecha_hora"]
    .astype(str)
    .str.replace('"', '', regex=False)
    .str.strip()
)

# Inicializar columna destino
df_general["fecha_hora"] = pd.NaT


# ========================
# 1. ISO YYYY-MM-DD HH:MM:SS
# ========================
mask_iso = fecha_raw.str.match(
    r"^\d{4}-\d{2}-\d{2} \d{2}:\d{2}:\d{2}$",
    na=False
)

df_general.loc[mask_iso, "fecha_hora"] = pd.to_datetime(
    fecha_raw[mask_iso],
    format="%Y-%m-%d %H:%M:%S",
    errors="coerce"
)
```

```python
# ========================
# 2. AM/PM MM-DD-YYYY
# ========================
mask_ampm = fecha_raw.str.contains(r'\bAM\b|\bPM\b', regex=True, na=False)

df_general.loc[mask_ampm, "fecha_hora"] = pd.to_datetime(
    fecha_raw[mask_ampm],
    format="%m-%d-%Y %I:%M %p",
    errors="coerce"
)


# ========================
# 3. AM/PM DD-MM-YYYY (fallback)
# ========================
mask_faltantes = mask_ampm & df_general["fecha_hora"].isna()

df_general.loc[mask_faltantes, "fecha_hora"] = pd.to_datetime(
    fecha_raw[mask_faltantes],
    format="%d-%m-%Y %I:%M %p",
    errors="coerce"
)


# ========================
# 4. Resto de formatos
# (DD/MM/YYYY, ISO corto, etc.)
# ========================
mask_rest = df_general["fecha_hora"].isna()

df_general.loc[mask_rest, "fecha_hora"] = pd.to_datetime(
    fecha_raw[mask_rest],
    dayfirst=True,
    errors="coerce"
)
```

```python
# ========================
# 5. Diagnóstico final
# ========================
print("Tipo final:", df_general["fecha_hora"].dtype)
print("NaT restantes:", df_general["fecha_hora"].isna().sum())

df_general["fecha_hora"].head(10)
```

11. Se limpia las columnas puesto quitando los guiones bajos.

```python
# "Limpiar columna Puesto"
df_general["puesto"]=(
    df_general["puesto"]
    .astype(str)
    .str.split("_")
    .str[0]
)

df_general["puesto"].unique()
```

```
array(['Observatorio', 'SantaFe', 'Pantitlan', 'RomaNorte', 'Lindavista',
       'Tlalpan', 'Tacubaya', 'Condesa', 'Coyoacan', 'Polanco', 'CU',
       'CentroHistorico', 'InsurgentesSur', 'IndiosVerdes', 'Reforma',
       'DelValle', 'Iztapalapa', 'Xochimilco'], dtype=object)
```

12. Homologación de los nombres.

```python
# Puesto
homologación_puesto={

    'SantaFe':'Santa Fe' ,
    'RomaNorte':'Roma Norte',
    'CU':'Ciudad Universidad',
    'CentroHistorico':'Centro Histórico',
    'LaPaz':'La Paz',
    'InsurgentesSur':'Insurgentes Sur',
    'IndiosVerdes':'Indios Verdes',
    'DelValle':'Del Valle'

    }

df_general["puesto"]=(
    df_general["puesto"]
    .replace(homologación_puesto)
)

# Homologar  Producto

homologación_producto={
    "Atole caliente":"Atole",
    "Cafe":"Café",
    "Rojito": "Tamal rojo",
    "Verdecito":"Tamal verde",
    "Rojo":"Tamal rojo",
    "Verde":"Tamal verde"
}


df_general["producto"]=(
    df_general["producto"]
    .replace(homologación_producto)
)

df_general.head(10)
```

	id_venta	fecha_hora	puesto	producto	categoría	canal	metodo_pago	precio
0	100001	2025-07-02 11:17:00	Observatorio	Tamal verde	Tamal	Pedido anticipado	Tarjeta	25.0
1	100002	2025-07-06 10:27:00	Santa Fe	Tamal verde	Tamal	Pedido anticipado	Tarjeta	25.0
2	100003	2025-08-05 07:45:00	Pantitlan	Tamal verde	Tamal	Puesto	Efectivo	25.0
3	100003	2025-08-05 07:45:00	Pantitlan	Bolillo	Complemento	Puesto	Efectivo	6.0
4	100003	2025-08-05 07:45:00	Pantitlan	Café	Bebida	Puesto	Efectivo	12.0
5	100004	2025-07-22 08:09:00	Roma Norte	Tamal verde	Tamal	Delivery	Efectivo	25.0
6	100004	2025-07-22 08:09:00	Roma Norte	Bolillo	Complemento	Delivery	Efectivo	6.0
7	100005	2025-08-24 08:38:00	Lindavista	Tamal rojo	Tamal	Pedido anticipado	Efectivo	26.0
8	100006	2025-07-06 10:18:00	Tlalpan	Tamal gourmet	Tamal	Pedido anticipado	Efectivo	35.0
9	100006	2025-07-06 10:18:00	Tlalpan	Bolillo	Complemento	Pedido anticipado	Efectivo	6.0

13. Convertí los datos a el tipo de dato que le corresponde.

```
#"id_venta"
df_general["id_venta"]=pd.to_numeric(df_general["id_venta"],errors="coerce").astype("Int64")

#puesto
df_general["puesto"]=(df_general['puesto'].astype(str).str.strip())

#producto
df_general["producto"]=(df_general['producto'].astype(str).str.strip())

#categoria
df_general["categoria"]=(df_general['categoria'].astype(str).str.strip())

#canal
df_general["canal"]=(df_general['canal'].astype(str).str.strip())

#metodo_pago
df_general["metodo_pago"]=(df_general['metodo_pago'].astype(str).str.strip())

#Precio

df_general["precio"]=pd.to_numeric(df_general["precio"],errors="coerce").astype("float64")
```

Guardar en un solo archivo llamado base general toda la información ya transformada y cargar al bucket.

```
ruta_salida ="/content/drive/MyDrive/Proyecto Google/MaterialesProyectoFinal/MaterialesProyectoFinal/csv_limpios/Base general.csv"
df_general.to_csv(
    ruta_salida,
    index=False,

)
```

Cargar al Bucket

```
cliente=storage.Client(project=project_ID)
bucket=cliente.bucket(BUCKET_NAME)
blob= bucket.blob(Destino)
blob.upload_from_filename(ruta_salida)

print("df_general cargado al bucket correctamente")
```

```
df_general cargado al bucket correctamente
```

Integración y análisis en la nube con SQL

Posteriormente, el conjunto de datos preparado en Google Colab fue almacenado en un **bucket de Google Cloud Storage** y vinculado a **BigQuery** mediante la creación de una tabla externa. Esta decisión técnica permitió consultar los datos directamente desde su ubicación de almacenamiento, facilitando una validación inicial de su estructura y contenido sin duplicar información.

Una vez verificada la integridad de los datos, se creó el esquema de trabajo correspondiente y se realizaron consultas exploratorias utilizando el identificador de venta (*id_venta*) como elemento central. Esta exploración confirmó que cada registro representa un artículo vendido dentro de una transacción.

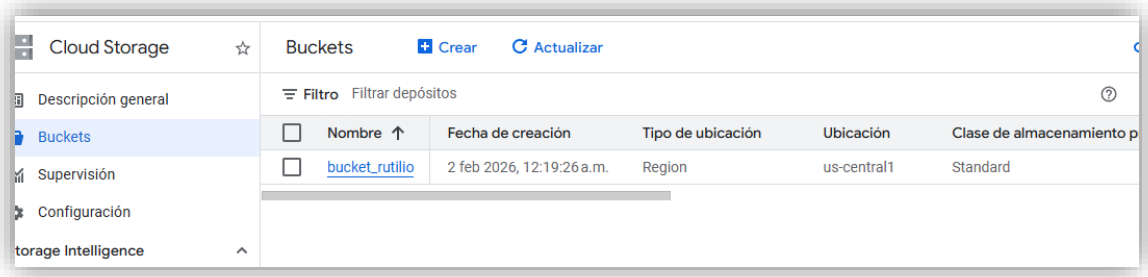
Transformación y construcción de la base final

Como parte del proceso de transformación, se construyó una base de datos final utilizando SQL en BigQuery. Mediante una subconsulta, se calculó la cantidad de productos vendidos por artículo dentro de cada venta, lo que permitió enriquecer la información transaccional con métricas relevantes para el análisis.

Posteriormente, se integraron estas métricas con los datos originales, conservando variables clave como fecha y hora de la venta, punto de venta, producto, categoría, canal de venta, método de pago y precio. El resultado fue una tabla final estructurada y optimizada para el análisis descriptivo, la visualización de resultados y la identificación de patrones de comportamiento del negocio.

Decisiones técnicas

El uso de **Google Colab** permitió un entorno flexible y reproducible para la preparación y limpieza de los datos, mientras que **BigQuery y SQL** facilitaron la integración, transformación y análisis eficiente de grandes volúmenes de información en la nube. Esta combinación de herramientas aseguró escalabilidad, consistencia y trazabilidad en todo el proceso analítico, sentando bases sólidas para la toma de decisiones estratégicas basadas en datos.



Creación y carga del bucket

Creación de tabla y carga del b... Ejecutar Guardar consulta Descargar

```
CREATE OR REPLACE EXTERNAL TABLE `tamalesdonrutilio.dataset.Base_General`
OPTIONS (

  format = 'csv',
  uris = ['gs://bucket_rutilio/datos/Base_general.csv']
);
```

Creación del schema

Creación de Schema Ejecutar Guardar consulta Descargar

```
Create schema if not exists `tamalesdonrutilio.dataset`
options(
  location="US"
);
```

Revisión de la Base General.

*Consult...ral *Base Final *Creació...ema

Consulta de la base general Ejecutar Guardar consulta

```
SELECT *
FROM `dataset.Base_General`
ORDER BY id_venta asc
```

Creación de la Base final

Base Fin... Ejecutar Guardar consulta

```sql
CREATE OR REPLACE TABLE `dataset.Base_Final` AS

WITH ventas_por_producto AS (
  SELECT
    id_venta AS Orden,
    producto,

    COUNT(*) AS cantidad

  FROM `dataset.Base_General`
  GROUP BY id_venta, producto
)

SELECT DISTINCT
  b.id_venta AS Orden,
  b.fecha_hora,
  b.puesto As Zona,
  b.producto,
  b.categoria,
  b.canal,
```

```sql
  b.metodo_pago,
  b.precio,
  v.cantidad
FROM `dataset.Base_General` b
LEFT JOIN ventas_por_producto v
  ON b.id_venta = v.Orden
 AND b.producto = v.producto
ORDER BY b.id_venta asc
```

Preguntas de negocio

El análisis de datos se orientó a responder las siguientes preguntas clave para la gestión del negocio:

- ¿Cuánto se vendió en el periodo analizado?
- ¿Cuánto se ganó a partir de las ventas realizadas?
- ¿En qué zonas se concentra el mayor volumen de ventas?
- ¿Cuál es el sabor o producto más vendido?
- ¿Qué categoría de productos genera mayores ingresos?
- ¿Cómo se comportan las ganancias de manera mensual?

Insights del análisis de datos

A partir del análisis de la información presentada en el tablero de control en Looker, se identificaron los siguientes hallazgos relevantes:

Volumen de ventas e ingresos

El negocio registró un total de 250,176 ventas, generando un ingreso acumulado de $4,411,972.00, lo que confirma un desempeño comercial sólido y consistente.

Ganancias mensuales

El comportamiento de las ganancias mensuales muestra variaciones a lo largo del periodo analizado. Se observa un incremento notable entre septiembre y octubre, meses en los que se alcanzan los niveles más altos de ingreso. Posteriormente, se presenta una disminución en noviembre y diciembre, lo cual sugiere la existencia de factores de estacionalidad o ajustes operativos que impactan la demanda. Este patrón resalta la importancia de planificar la producción y el inventario de forma anticipada.

Zonas con mayor venta

El análisis por zona indica que Ciudad concentra el mayor volumen de ventas, seguida por Santa Fe, posicionándose como los principales puntos de generación de ingresos. Otras zonas presentan un menor desempeño relativo.

Producto (sabor) más vendido

La guajolota es el producto con mayor volumen de ventas, seguida por los tamales verde y rojo, lo que evidencia una preferencia clara por sabores tradicionales.

Categoría más vendida

La categoría Tamal es la principal generadora de ingresos, con una contribución significativamente mayor en comparación con Complementos y Bebidas, las cuales funcionan como productos adicionales que incrementan el valor de cada transacción.

Métodos de pago

La distribución de los métodos de pago se encuentra equilibrada entre efectivo, transferencia y tarjeta, reflejando una operación flexible y alineada con las preferencias de los clientes.

Conclusiones

El análisis de datos demuestra que la Tamalería *Don Rutilio* opera de manera rentable y con un portafolio de productos bien definido, donde los tamales tradicionales constituyen el principal motor de ingresos. La variación en las ganancias mensuales y la concentración de ventas por zona y producto evidencian oportunidades claras para mejorar la planeación operativa, optimizar inventarios y fortalecer la estrategia comercial.

Recomendaciones

- Priorizar la producción de los productos más vendidos, especialmente guajolotas y tamales tradicionales, para evitar pérdidas por desabasto.
- Fortalecer la operación en las zonas con mayor demanda, como Ciudad Universidad y Santa Fe, evaluando la posibilidad de ampliar horarios, inventario o incluso puntos de venta.
- Diseñar estrategias de impulso para las zonas con menor desempeño, mediante promociones, ajustes en la oferta de productos o campañas focalizadas.
- Aprovechar los complementos y bebidas como estrategia para incrementar el ticket promedio por venta.
- Mantener y optimizar los métodos de pago disponibles, asegurando una experiencia de compra ágil, accesible y alineada con las preferencias de los clientes.
- Monitorear de forma continua las ganancias mensuales a través del visor de Looker, permitiendo realizar ajustes oportunos en producción, precios y estrategias comerciales.
- Incrementar las ventas en los meses de noviembre y diciembre aprovechando los días festivos como: Día de muerto, la virgen de guadalupe, Navidad, Noche buena, posadas.
-

En conjunto, estos insights permiten a Rutilio tomar decisiones basadas en datos, mejorar el control operativo y financiero, y sentar bases sólidas para un crecimiento sostenible y ordenado del negocio.

Referencia:

Enlace para visualizar el visor.

https://lookerstudio.google.com/reporting/7d6cd39c-e777-41d5-8d16-bd29aca543c5

Big Query

https://console.cloud.google.com/bigquery?ws=!1m7!1m6!12m5!1m3!1stamalesdonrutilio!2sus-central1!3s57597089-ac12-44dd-b43e-0f2ad04754e9!2e1

Colab

https://colab.research.google.com/drive/1y0Zi_1f8V-5lLbFoG2tNQUyHKuV74OuK?usp=sharing

Video

https://www.youtube.com/watch?v=xaLfsQl84dU